

PRÁCTICA DE SISTEMAS OPERATIVOS

5 variantes de control: Signalización, Message Passing, Pipe, fork() y execv()

Guía de entrenamiento basada en la estructura del control proporcionado.

Objetivo: practicar cómo combinar señales, jerarquía de procesos e IPC en ejercicios que cambian ligeramente respecto al modelo original.

Idea clave: no memorices un código único. Aprende a reconocer qué mecanismo corresponde a cada parte del enunciado.

Mapa mental

Concepto	Qué hace
fork()	Crea un nuevo proceso.
execv()	Reemplaza la imagen/programa del proceso actual por otro ejecutable.
signal()	Asocia un manejador a una señal.
kill -2 PID	Envía SIGINT al proceso indicado.
kill -5 PID	Envía SIGTRAP al proceso indicado.
pipe()	Crea una tubería para comunicación mediante descriptores de lectura/escritura.
write()	Escribe datos en el Pipe.
read()	Lee datos del Pipe.
msgsnd()	Envía un mensaje a una cola System V.
msgrcv()	Recibe un mensaje desde una cola System V.
pause()	Deja al proceso esperando una señal.

EJERCICIO 1 — EL MÁS PARECIDO AL CONTROL

Crear un programa en C con tres procesos:

- A es el padre.
- A crea B mediante `fork()`.
- B crea C mediante `fork()` y C debe convertirse en `proceso_c` mediante `execv()`.
- A captura `SIGINT` (2) y `SIGTRAP` (5).
- `SIGINT`: A envía un mensaje a B mediante Message Queue.
- `SIGTRAP`: A envía un mensaje a C mediante Pipe.
- B y C imprimen el mensaje recibido y continúan esperando.

Estructura:

A --fork--> B --fork/execv--> C

`SIGINT`: A --Queue--> B

`SIGTRAP`: A --Pipe--> C

Especificaciones que debes identificar

- `fork()`: A crea B y B crea el proceso que se convertirá en C.
- `execv()`: el clon de B ejecuta `./proceso_c`.
- `SIGINT` (2): Message Queue hacia B.
- `SIGTRAP` (5): Pipe hacia C.

Solución

`proceso_a.c`

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>

#define TAM 100

struct mensaje {
    long tipo;
    char texto[TAM];
};

int pipefd[2];
int cola;

void manejar_sigint(int signo)
{
    struct mensaje msg;
    msg.tipo = 1;
    strcpy(msg.texto, "Hola B, mensaje recibido mediante Message Queue");
    printf("\n[A] Recibi SIGINT\n");
    msgsnd(cola, &msg, sizeof(msg.texto), 0);
}

void manejar_sigtrap(int signo)
{
    char mensaje[TAM] = "Hola C, mensaje recibido mediante Pipe";
    printf("\n[A] Recibi SIGTRAP\n");
    write(pipefd[1], mensaje, strlen(mensaje) + 1);
}

int main()
{
    pid_t pid_b;

    pipe(pipefd);
    cola = msgget(IPC_PRIVATE, 0666 | IPC_CREAT);
```

```

signal(SIGINT, manejar_sigint);
signal(SIGTRAP, manejar_sigtrap);

pid_b = fork();

if (pid_b == 0) {

    pid_t pid_c = fork();

    if (pid_c == 0) {
        char fd[20];
        sprintf(fd, "%d", pipefd[0]);

        char *args[] = {"/proceso_c", fd, NULL};
        execv("/proceso_c", args);

        perror("execv");
        exit(1);
    }

    close(pipefd[1]);

    struct mensaje msg;

    while (1) {
        msgrcv cola, &msg, sizeof(msg.texto), 1, 0);

        printf("\n[B] PID: %d\n", getpid());
        printf("[B] Mensaje: %s\n", msg.texto);
    }

    close(pipefd[0]);

    printf("[A] PID: %d\n", getpid());
    printf("kill -2 %d\n", getpid());
    printf("kill -5 %d\n", getpid());

    while (1)
        pause();

    return 0;
}

```

proceso_c.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define TAM 100

int main(int argc, char *argv[])
{
    int fd;
    char mensaje[TAM];

    if (argc != 2) {
        printf("Falta descriptor\n");
        return 1;
    }

    fd = atoi(argv[1]);

    while (1) {
        int n = read(fd, mensaje, TAM);

        if (n > 0) {
            mensaje[n] = '\0';
            printf("\n[C] PID: %d\n", getpid());
            printf("[C] Mensaje por Pipe: %s\n", mensaje);
        }

        return 0;
    }
}

```

Compilación y prueba

```

gcc proceso_c.c -o proceso_c
gcc proceso_a.c -o proceso_a
./proceso_a

```

En otra terminal, usa el PID que imprime A:

```
kill -2 PID_A  
kill -5 PID_A
```

Qué debes poder explicar: quién ejecuta cada bloque después de cada `fork()`, qué descriptor del Pipe usa cada proceso, qué señal activa cada manejador y qué proceso recibe cada mensaje.

EJERCICIO 2 — INTERCAMBIAR LOS IPC

Crear A, B y C.

- A crea B.
- B crea C mediante `fork()` + `execv()`.
- `SIGINT` debe enviar un mensaje mediante Pipe a C.
- `SIGTRAP` debe enviar un mensaje mediante Message Queue a B.
- B y C imprimen su PID y el mensaje recibido.
- Ambos continúan esperando.

Flujo:

`SIGINT` -> A -> Pipe -> C

`SIGTRAP` -> A -> Queue -> B

Especificaciones que debes identificar

- `fork()`: A crea B; B crea C.
- `execv()`: C recibe el descriptor del Pipe por `argv`.
- `SIGINT` (2): Pipe hacia C.
- `SIGTRAP` (5): Message Queue hacia B.

Solución

proceso_a.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>

#define TAM 100

struct mensaje {
    long tipo;
    char texto[TAM];
};

int pipefd[2];
int cola;

void sigint(int signo)
{
    char mensaje[] = "SIGINT: mensaje de A para C mediante Pipe";
    printf("[A] SIGINT recibida\n");
    write(pipefd[1], mensaje, strlen(mensaje) + 1);
}

void sigtrap(int signo)
{
    struct mensaje msg;
    msg.tipo = 1;
    strcpy(msg.texto, "SIGTRAP: mensaje de A para B mediante Queue");
    printf("[A] SIGTRAP recibida\n");
    msgsnd(cola, &msg, sizeof(msg.texto), 0);
}

int main()
{
    pipe(pipefd);
    cola = msgget(IPC_PRIVATE, 0666 | IPC_CREAT);

    signal(SIGINT, sigint);
    signal(SIGTRAP, sigtrap);

    pid_t b = fork();
```

```

if (b == 0) {
    pid_t c = fork();

    if (c == 0) {
        char fd[20];
        sprintf(fd, "%d", pipefd[0]);

        char *args[] = {"/proceso_c", fd, NULL};
        execv("/proceso_c", args);

        perror("execv");
        exit(1);
    }

    close(pipefd[1]);

    struct mensaje msg;

    while (1) {
        msgrcv(cola, &msg, sizeof(msg.texto), 1, 0);

        printf("\n[B] PID %d recibió: %s\n",
            getpid(), msg.texto);
    }

    close(pipefd[0]);

    printf("[A] PID = %d\n", getpid());

    while (1)
        pause();

    return 0;
}

```

proceso_c.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    int fd;
    char mensaje[100];

    if (argc != 2)
        return 1;

    fd = atoi(argv[1]);

    while (1) {
        int n = read(fd, mensaje, 100);

        if (n > 0) {
            mensaje[n] = '\0';

            printf("\n[C] PID %d recibió por Pipe: %s\n",
                getpid(), mensaje);
        }
    }

    return 0;
}

```

Compilación y prueba

```

gcc proceso_c.c -o proceso_c
gcc proceso_a.c -o proceso_a
./proceso_a

```

```

# En otra terminal, usa el PID que imprime A:
kill -2 PID_A
kill -5 PID_A

```

Qué debes poder explicar: quién ejecuta cada bloque después de cada fork(), qué descriptor del Pipe usa cada proceso, qué señal activa cada manejador y qué proceso recibe cada mensaje.

EJERCICIO 3 — B REENVÍA EL MENSAJE A C

Crear A, B y C en cadena.

- A recibe SIGINT.
- A envía un mensaje a B mediante Message Queue.
- B recibe el mensaje, lo modifica y lo envía a C mediante Pipe.
- C se crea mediante fork() + execv().
- C imprime el mensaje final.
- B y C continúan esperando.

Flujo:

SIGINT -> A -> Queue -> B -> Pipe -> C

Especificaciones que debes identificar

- SIGINT: A envía a B por cola.
- B recibe y transforma el mensaje.
- B escribe el resultado en el Pipe.
- C recibe por Pipe y fue creado mediante execv().

Solución

proceso_a.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>

struct mensaje {
    long tipo;
    char texto[100];
};

int cola;

void manejar(int signo)
{
    struct mensaje msg;
    msg.tipo = 1;
    strcpy(msg.texto, "Mensaje original enviado por A");

    printf("[A] Señal recibida\n");

    msgsnd(cola, &msg, sizeof(msg.texto), 0);
}

int main()
{
    int pipefd[2];
    pipe(pipefd);

    cola = msgget(IPC_PRIVATE, 0666 | IPC_CREAT);
    signal(SIGINT, manejar);

    pid_t b = fork();

    if (b == 0) {
        pid_t c = fork();

        if (c == 0) {
            char fd[20];
            sprintf(fd, "%d", pipefd[0]);

            char *args[] = { "./proceso_c", fd, NULL };
            execv("./proceso_c", args);

            perror("execv");
        }
    }
}
```

```

        exit(1);
    }

    struct mensaje msg;
    char nuevo[100];

    while (1) {
        msgrcv(cola, &msg, sizeof(msg.texto), 1, 0);

        printf("[B] Recibí: %s\n", msg.texto);

        sprintf(nuevo,
            "B modificó el mensaje: [%s]",
            msg.texto);

        write(pipefd[1], nuevo, strlen(nuevo) + 1);
    }

    printf("[A] PID = %d\n", getpid());
    printf("Ejecuta: kill -2 %d\n", getpid());

    while (1)
        pause();

    return 0;
}

```

proceso_c.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    int fd;
    char mensaje[100];

    if (argc != 2)
        return 1;

    fd = atoi(argv[1]);

    while (1) {
        int n = read(fd, mensaje, sizeof(mensaje));

        if (n > 0) {
            mensaje[n] = '\0';

            printf("[C] PID = %d\n", getpid());
            printf("[C] Mensaje final: %s\n", mensaje);
        }
    }

    return 0;
}

```

Compilación y prueba

```

gcc proceso_c.c -o proceso_c
gcc proceso_a.c -o proceso_a
./proceso_a

```

```

# En otra terminal, usa el PID que imprime A:
kill -2 PID_A
kill -5 PID_A

```

Qué debes poder explicar: quién ejecuta cada bloque después de cada fork(), qué descriptor del Pipe usa cada proceso, qué señal activa cada manejador y qué proceso recibe cada mensaje.

EJERCICIO 4 — DOS HIJOS DE A

Cambiar la jerarquía a:

A
/\n
B C

- A crea B mediante `fork()`.
- A crea C mediante otro `fork()`.
- `SIGINT` -> A envía mensaje a B mediante Message Queue.
- `SIGTRAP` -> A envía mensaje a C mediante Pipe.
- B y C imprimen su PID.
- Ambos continúan esperando.

En esta variante NO se requiere `execv()`, porque C es creado directamente como hijo de A.

Especificaciones que debes identificar

- A crea B y C directamente.
- `SIGINT` (2): Message Queue hacia B.
- `SIGTRAP` (5): Pipe hacia C.
- No se necesita `execv()` en esta variante.

Solución

proceso.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>

struct mensaje {
    long tipo;
    char texto[100];
};

int pipefd[2];
int cola;

void sigint(int signo)
{
    struct mensaje msg;
    msg.tipo = 1;
    strcpy(msg.texto, "Mensaje para B");

    msgsnd(cola, &msg, sizeof(msg.texto), 0);
}

void sigtrap(int signo)
{
    char mensaje[] = "Mensaje para C";

    write(pipefd[1],
            mensaje,
            strlen(mensaje) + 1);
}

int main()
{
    pipe(pipefd);

    cola = msgget(IPC_PRIVATE, 0666 | IPC_CREAT);

    signal(SIGINT, sigint);
    signal(SIGTRAP, sigtrap);
```

```

pid_t b = fork();

if (b == 0) {
    struct mensaje msg;

    close(pipefd[0]);
    close(pipefd[1]);

    while (1) {
        msgrcv cola, &msg, sizeof(msg.texto), 1, 0);

        printf("[B] PID %d: %s\n",
               getpid(), msg.texto);
    }
}

pid_t c = fork();

if (c == 0) {
    char mensaje[100];

    close(pipefd[1]);

    while (1) {
        int n = read(pipefd[0], mensaje, 100);

        if (n > 0) {
            mensaje[n] = '\0';

            printf("[C] PID %d: %s\n",
                   getpid(), mensaje);
        }
    }

    close(pipefd[0]);

    printf("[A] PID = %d\n", getpid());
    printf("kill -2 %d -> B\n", getpid());
    printf("kill -5 %d -> C\n", getpid());

    while (1)
        pause();

    return 0;
}

```

Compilación y prueba

```

gcc proceso.c -o proceso
./proceso

```

```

# En otra terminal:
kill -2 PID_A
kill -5 PID_A

```

Qué debes poder explicar: quién ejecuta cada bloque después de cada fork(), qué descriptor del Pipe usa cada proceso, qué señal activa cada manejador y qué proceso recibe cada mensaje.

EJERCICIO 5 — JEFE FINAL

Crear A, B y C con:

```
A
|
fork()
|
B
|
fork()
|
C
```

Requisitos:

- A captura SIGINT y SIGTRAP.
- SIGINT: A -> Message Queue -> B.
- B modifica el mensaje y lo envía por Pipe -> C.
- SIGTRAP: A -> Pipe -> C directamente.
- C debe ser creado mediante fork() + execv().
- B y C continúan funcionando después de recibir mensajes.

Este ejercicio combina los mecanismos en una misma solución.

Especificaciones que debes identificar

- A captura dos señales.
- SIGINT: A → Queue → B → Pipe → C.
- SIGTRAP: A → Pipe → C.
- C recibe el descriptor mediante argv y execv().

Solución

proceso_a.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>

struct mensaje {
    long tipo;
    char texto[200];
};

int pipefd[2];
int cola;

void sigint(int signo)
{
    struct mensaje msg;

    msg.tipo = 1;
    strcpy(msg.texto,
        "Mensaje de A recibido por B");

    printf("\n[A] SIGINT\n");

    msgsnd(cola,
        &msg,
        sizeof(msg.texto),
        0);
}
```

```

void sigtrap(int signo)
{
    char mensaje[] =
        "Mensaje directo de A hacia C";

    printf("\n[A] SIGTRAP\n");

    write(pipefd[1],
        mensaje,
        strlen(mensaje) + 1);
}

int main()
{
    pipe(pipefd);

    cola = msgget(IPC_PRIVATE,
        0666 | IPC_CREAT);

    signal(SIGINT, sigint);
    signal(SIGTRAP, sigtrap);

    pid_t b = fork();

    if (b == 0) {
        pid_t c = fork();

        if (c == 0) {
            char fd[20];

            sprintf(fd,
                "%d",
                pipefd[0]);

            char *args[] = {
                "./proceso_c",
                fd,
                NULL
            };

            execv("./proceso_c",
                args);

            perror("execv");
            exit(1);
        }

        struct mensaje msg;
        char nuevo[200];

        while (1) {
            msgrcv(cola,
                &msg,
                sizeof(msg.texto),
                1,
                0);

            printf("\n[B] PID = %d\n",
                getpid());

            printf("[B] Recibido: %s\n",
                msg.texto);

            sprintf(nuevo,
                "%s | Procesado por B (PID %d)",
                msg.texto,
                getpid());

            write(pipefd[1],
                nuevo,
                strlen(nuevo) + 1);
        }
    }

    printf("[A] PID = %d\n",
        getpid());

    printf("\nSIGINT -> A -> Queue -> B -> Pipe -> C\n");
    printf("SIGTRAP -> A -> Pipe -> C\n");

    printf("\nkill -2 %d\n", getpid());
    printf("kill -5 %d\n", getpid());
}

```

```

        while (1)
            pause();

        return 0;
}

```

proceso_c.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    int fd;
    char mensaje[200];

    if (argc != 2) {
        printf("Uso: proceso_c <descriptor>\n");
        return 1;
    }

    fd = atoi(argv[1]);

    printf("[C] Soy C. PID = %d\n",
        getpid());

    while (1) {
        int n = read(fd,
            mensaje,
            sizeof(mensaje));

        if (n > 0) {
            mensaje[n] = '\0';

            printf("\n[C] PID = %d\n",
                getpid());

            printf("[C] Recibido por PIPE:\n");
            printf("[C] %s\n", mensaje);
        }
    }

    return 0;
}

```

Compilación y prueba

```

gcc proceso_c.c -o proceso_c
gcc proceso_a.c -o proceso_a
./proceso_a

```

```

# En otra terminal, usa el PID que imprime A:
kill -2 PID_A
kill -5 PID_A

```

Qué debes poder explicar: quién ejecuta cada bloque después de cada fork(), qué descriptor del Pipe usa cada proceso, qué señal activa cada manejador y qué proceso recibe cada mensaje.

CHECKLIST FINAL PARA EL CONTROL

Cuando te den un enunciado nuevo, identifica estas seis cosas antes de escribir código:

1. ¿Quién crea a quién? → dibuja la jerarquía de `fork()`.
2. ¿Qué proceso recibe cada señal? → identifica el PID objetivo de `kill`.
3. ¿Qué IPC se usa para cada comunicación? → Pipe o Message Queue.
4. ¿Dónde aparece `execv()`? → recuerda que reemplaza el programa del proceso que lo ejecuta.
5. ¿Qué descriptor viaja por `argv`? → normalmente el extremo de lectura del Pipe para C.
6. ¿Qué procesos deben continuar esperando? → usa ciclos de espera adecuados.

Comandos básicos

```
gcc proceso_c.c -o proceso_c
gcc proceso_a.c -o proceso_a
./proceso_a
```

```
ps aux | grep proceso_a
kill -2 PID
kill -5 PID
```

Referencia al control original

El control proporcionado plantea una estructura de tres procesos A, B y C, captura de señales 2 y 5, Message Passing mediante cola, Pipe, `fork()` y `execv()`. [filecite\turn0file0\L24-L47](#)

Nota de estudio: los cinco ejercicios son variantes de práctica elaboradas a partir de esa estructura. Son simulaciones para entrenar la resolución, no una reproducción del control.